```
// Name: Utkarsha Patil
// Roll no: SEA52
// Binary Search Tree Operations for Online Directory Systems

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct BST{
   char name[10], phone[11];
   struct BST *left, *right;
}BST;
void main(){
   BST *root = NULL, *nw, *temp;
   int op, ans;
   char key[10];
   void insert(BST *, BST *);
   void inorder(BST *);
   void preorder(BST *);
   void postorder(BST *);
   void search(BST *, char *);
   do{
      printf("\n1. Create\n2. Traverse\n3. Search\n4. Exit\n");
      printf("Enter option: ");
      scanf("%d", &op);
      switch(op){
      case 1:
         do{
            nw = (BST*)malloc(sizeof(BST));
```

```
      printf("Enter Contact's name: ");
      scanf("%s", nw->name);
      printf("Enter Contact's phone: ");
      scanf("%s", nw->phone);
      nw->left = NULL;
      nw->right = NULL;
      if(root == NULL){
        root = nw;
      }else{
        insert(root, nw);
      }
      printf("Do you want to continue? : ");
      scanf("%d", &ans);
    }while(ans == 1);
    break;
  case 2:
    if(root == NULL){
      printf("Tree is not created\n");
    }else{
      printf("Travrsals for given tree are: \n");
      printf("Inorder: ");
      inorder(root);
      printf("\nPreorder: ");
      preorder(root);
      printf("\nPostorder: ");
      postorder(root);
    }
    break;
```

```
      case 3:
        if(root == NULL){
          printf("Tree is not created\n");
        }else{
          printf("Enter contact name to search: ");
          scanf("%s", key);
          search(root, key);
        }
        break;
      case 4:
        printf("Exited Successfully\n");
        break;
      default:
        printf("Invalid Option\n");
      }
   }while(op != 4);
}
void insert(BST *root, BST *nw){
   if(strcasecmp(nw->name, root->name) < 0){
      if(root->left == NULL){
        root->left = nw;
      }else{
        insert(root->left, nw);
      } }
   if(strcasecmp(nw->name, root->name) > 0){
      if(root->right == NULL){
        root->right = nw;
```

```
        }else{
            insert(root->right, nw);
        }}}
void inorder(BST *temp){
    if(temp != NULL){
        inorder(temp->left);
        printf("%s ", temp->name);
        inorder(temp->right);
    }}
void preorder(BST *temp){
    if(temp != NULL){
        printf("%s ", temp->name);
        preorder(temp->left);
        preorder(temp->right);
    }}
void postorder(BST *temp){
    if(temp != NULL){
        postorder(temp->left);
        postorder(temp->right);
        printf("%s ", temp->name);
    }
    printf("\n");
}
void search(BST *root, char *key){
    if(root == NULL){
        printf("\nContact not found");
        return;
    }
```

```
  if(strcasecmp(key, root->name) == 0){
     printf("Contact found");
     printf("\nName: %s", root->name);
     printf("\nPhone: %s\n", root->phone);
  }
  else if(strcasecmp(key, root->name) < 0){
     search(root->left, key);
  }
  else{
     search(root->right, key);
  }}
//Output

1. Create
2. Traverse
3. Search
4. Exit
Enter option: 1
Enter Contact's name: vaishnavi
Enter Contact's phone: 123456
Do you want to continue? : 1
Enter Contact's name: arya
Enter Contact's phone: 789845
Do you want to continue? : 1
Enter Contact's name: utkarsha
Enter Contact's phone: 564983
Do you want to continue? : 1
```

Enter Contact's name: purva

Enter Contact's phone: 653149

Do you want to continue? : 0

1. Create

2. Traverse

3. Search

4. Exit

Enter option: 2

Travrsals for given tree are:

Inorder: arya purva utkarsha vaishnavi

Preorder: vaishnavi arya utkarsha purva

Postorder: purva utkarsha arya vaishnavi

1. Create

2. Traverse

3. Search

4. Exit

Enter option: 3

Enter contact name to search: purva

Contact found

Name: purva

Phone: 653149

1. Create

2. Traverse

3. Search

4. Exit

Enter option: 4

Exited Successfully